National Education Society®

JNN College of Engineering, Shivamogga

Department of Information Science and Engineering

Internship Project Report

Banking System Web Application

Django Backend | Stitch UI | REST API | SQLite Database

Login/Register | View Accounts | Transfers | Transaction History

Prepared by:

Name of the Student: Abhishek Gowda

USN: _____

External Guide: _____

Internal Guide: _____

Internship Duration: 15 April 2026 - 16 April 2026

Organization: ABCDEFF Corporation Pvt. Ltd., Bangalore

Project Theme

A secure dark-mode banking application that demonstrates authentication, account management, transfer workflows, transaction pages, summary views, and REST API endpoints for banking operations.

Contents

Roadmap of the report

Introduction

Project overview

- The Banking System Web Application is a full-stack Django project focused on digital banking workflows.
- The application provides login and registration, account viewing, account details, account creation/editing, account summary, money transfer, transaction entry, and a no-account empty state.
- The project combines a modern Stitch-generated dark UI with Django views, URL routing, custom user authentication, banking models, and REST framework APIs.

Objectives

What the project was built to achieve

- Create a realistic banking dashboard experience with a premium dark-mode interface.
- Implement clear navigation from login to dashboard, accounts, account details, transfer, summary, and transactions.
- Design backend URL routes that map cleanly to frontend templates.
- Store account and transaction data using Django models.
- Expose API endpoints for account listing, account details, deposits, withdrawals, transfers, and transaction history.
- Debug frontend routing, pagination behavior, and no-account page behavior.

Technology Stack

Tools and frameworks used

Frontend

Stitch templates, Tailwind CSS, Material Symbols, responsive dark banking UI.

Backend

Django views, URL routing, custom user model, SQLite database.

API

Django REST Framework endpoints for accounts, details, transfers, deposit, withdraw.

Tools

VS Code, Chrome, local development server, Python scripts for debugging.

- Frontend: HTML templates, Stitch-generated UI, Tailwind CSS CDN, Material Symbols icons, responsive dark-mode layout.
- Backend: Python, Django, Django views, Django URL routing, custom user model, SQLite database.
- API Layer: Django REST Framework viewsets, APIView, router endpoints, authenticated account actions.
- Database: SQLite during development with User, BankAccount, and Transaction models.
- Development Tools: VS Code, Chrome browser, local Django server at 127.0.0.1:8000.

Frontend Work Completed

Screens implemented in the templates folder

- login.html: secure login page that posts phone and password to the backend.
- register.html: registration page with name, email, phone, password, and CSRF setup.
- accounts.html: view accounts page with account cards, details buttons, edit buttons, and add-account navigation.
- account_details.html: account overview page with balance, account number, IFSC, transaction history, add-transaction button, and working pagination.
- add_account.html: form for adding a new account with account number, account type, IFSC, and initial balance.
- edit_account.html: page for editing account details/preferences.
- summary.html: combined account summary view.
- transfer.html: transfer money workflow screen.
- transactions.html: add transaction page.
- no_account.html: empty-state page for users with no accounts.

Backend Work Completed

Django views and routes

- Created explicit route mapping in banking_system/urls.py for each page.
- Login and registration are handled through users.views.
- Dashboard redirects to /accounts/ for a clean post-login flow.
- View Accounts uses /accounts/ and renders accounts.html.
- Account details use /accounts/<account_id>/ and render account_details.html.
- Edit account uses /accounts/<account_id>/edit/ or /edit-account/.
- Add account supports GET for the form and POST for creating BankAccount rows.
- No-account page is available at /no-account/.
- Summary, transfer, and transactions each have dedicated routes.

Database Design

Core models used by the project

- User: custom Django user model using email as username field, with name and phone fields.
- BankAccount: linked to a user, stores account_number, account_type, and balance.
- Transaction: linked to a bank account, stores transaction_type, amount, and created_at timestamp.
- The design supports one user having multiple bank accounts and each account having multiple transactions.

REST API Work

Banking API behavior

- GET /api/bank_accounts/ returns authenticated user bank accounts.
- GET /api/account-details/?account_id=<id> returns account details and transaction list.
- ViewSet route /api/accounts/ supports account CRUD for authenticated users.
- POST /api/accounts/<id>/deposit/ validates amount, updates balance, and creates a deposit transaction.
- POST /api/accounts/<id>/withdraw/ validates funds, updates balance, and creates a withdraw transaction.
- POST /api/accounts/<id>/transfer/ moves money between accounts and creates debit/credit transaction records.
- GET /api/accounts/transaction_history/ returns the user transaction history.

User Flow

Navigation flow after debugging

- User opens / or /login/ and signs in.
- Successful login redirects to /dashboard/, which redirects to /accounts/.
- Clicking View Accounts opens accounts.html.
- Clicking any account card opens account_details.html through /accounts/<id>/.
- Clicking Add Account opens add_account.html.
- Clicking edit icon opens edit_account.html.
- Clicking Account Summary opens summary.html.
- Clicking Transfer Money opens transfer.html.
- Clicking Add Transaction opens transactions.html.
- Logout returns to /login/.

Debugging and Improvements

Issues fixed during development

- Resolved incorrect login form action that opened the wrong page.
- Separated login/register routes from banking page routes.
- Fixed View Accounts so it opens accounts.html instead of add-account/no-account unexpectedly.
- Allowed static sample account cards to open account_details.html while database data is still being connected.
- Fixed transaction pagination UI where Add Transaction text appeared inside the page number button.
- Implemented working frontend pagination for transaction rows.
- Cleaned backend URL mapping so each page has one clear URL.

Final URL Map

Important local development URLs

- / or /login/ : Login page
- /register/ : Register page
- /dashboard/ : Redirects to accounts
- /accounts/ : View accounts page
- /accounts/1/ : Account details page
- /accounts/1/edit/ : Edit account page
- /add-account/ : Add account page
- /summary/ : Account summary page
- /transfer/ : Transfer money page
- /transactions/ : Add transaction page
- /no-account/ : No account empty state
- /api/bank_accounts/ : Authenticated bank accounts API
- /api/account-details/?account_id=1 : Account details API

Conclusion

Outcome of the internship project

- The project demonstrates a complete banking web application foundation with a polished frontend and functional Django backend structure.
- It includes authentication pages, account navigation, account management, transfer UI, transaction UI, summary pages, empty states, and REST API endpoints.
- The system is ready for the next development stage: connecting all template forms to database operations, adding real transaction history rendering, and improving authentication/session handling.

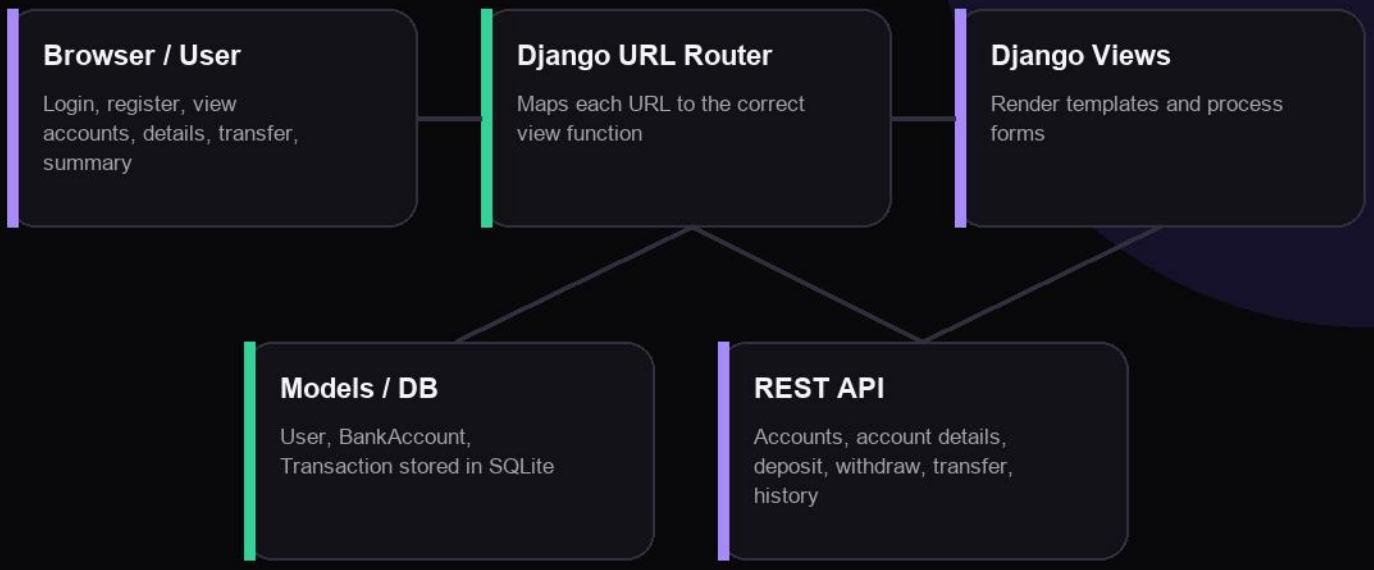
References

Resources and sources used

- Django official documentation for views, URL routing, models, and authentication.
- Django REST Framework documentation for ViewSet, APIView, routers, actions, and permissions.
- Tailwind CSS documentation for utility-first UI styling.
- Google Material Symbols for interface icons.
- Stitch-generated frontend templates adapted for the banking system user interface.

System Architecture

High-level structure of the Django banking system



Architecture Notes

- The browser communicates with Django views through clean page URLs.
- The views render Stitch/Tailwind templates and handle form submissions.
- Banking operations are represented in models and also exposed through API endpoints.
- Authentication protects API banking data using the logged-in user context.

Selected Implementation Snapshot

Representative backend structures used in the project

```
class BankAccount(models.Model):
    user = models.ForeignKey(settings.AUTH_USER_MODEL, on_delete=models.CASCADE)
    account_number = models.CharField(max_length=20)
    account_type = models.CharField(max_length=20)
    balance = models.DecimalField(max_digits=10, decimal_places=2, default=0)

urlpatterns = [
    path('login/', user_views.login_page),
    path('accounts/', views.accounts),
    path('accounts/<int:account_id>/', views.account_details),
    path('add-account/', views.add_account),
    path('summary/', views.summary_page),
    path('transfer/', views.transfer_page),
    path('transactions/', views.transactions_page),
]
```

This snapshot shows how the project connects database entities and page routes. The complete project includes separate templates for every major banking workflow and API views for core banking actions.